

CS121 Data Structures in C

12 Sorting - 2

Outline

- Exchange sort
 - ♦ Bubble sort
 - ♦ Quick sort
- Worst case lower bound for comparison-based sorting
- Non-comparison based sorting
 - ♦ Bucket sort
 - ♦ Radix sort
- Summary of sorting algorithms

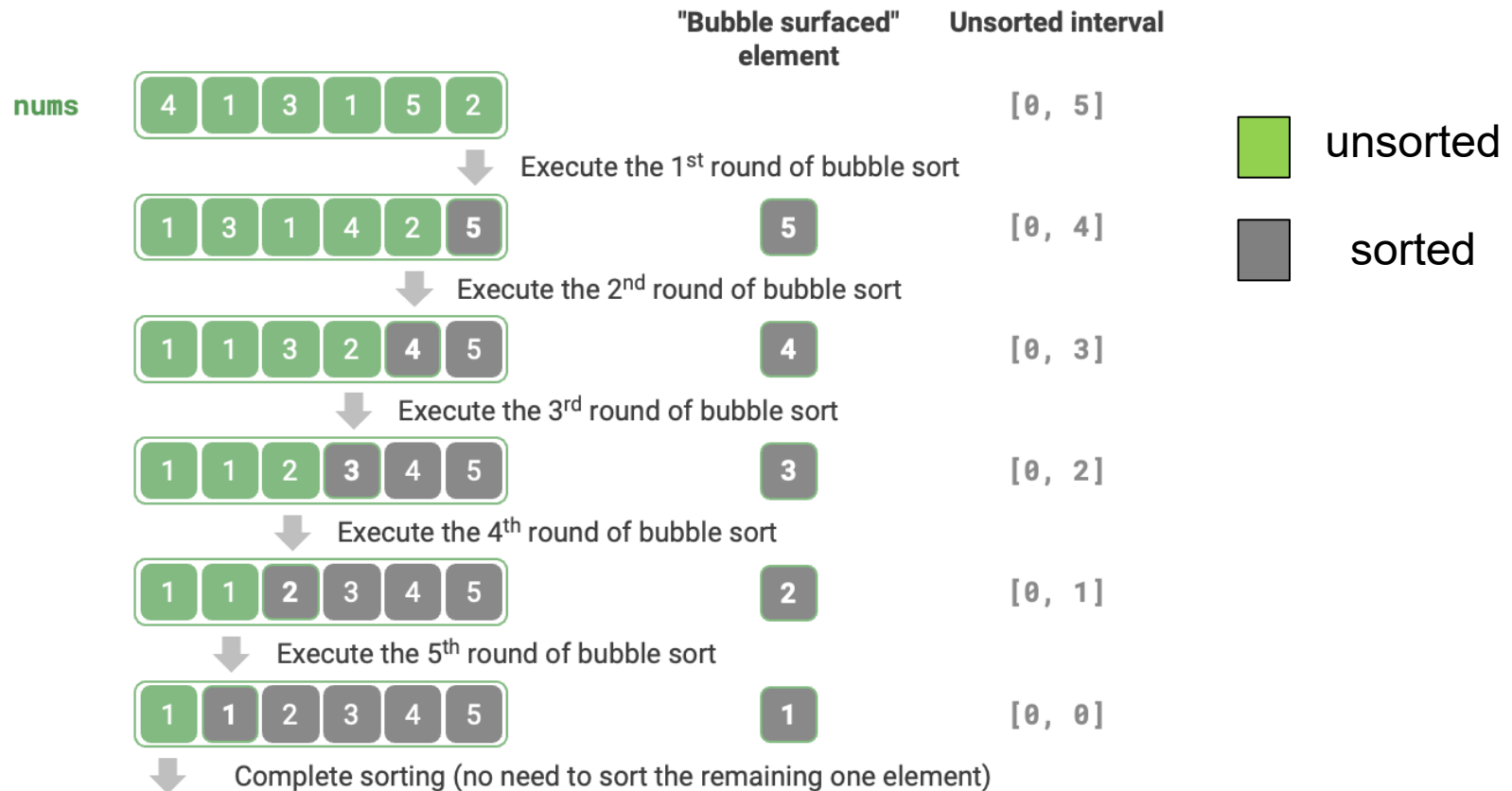
Bubblesort

Exchange Sort

- In exchange sorts we exchange elements that are out of order until the entire list is sorted.
 - ♦ Bubble sort
 - ♦ Quick sort
- Bubble sort
 - ♦ The list in any moment is divided into two sublists: unsorted and sorted.
 - ♦ The largest element (in the unsorted sublist) is bubbled from the unsorted sublist and moved to the sorted sublist.

Exchange Sort

- Bubble sort
 - ♦ The largest element (in the unsorted sublist) is bubbled from the unsorted sublist and moved to the sorted sublist.



Bubble Sort Example and Analysis

- The best case: sorted order
 - ♦ Only one pass and $n-1$ comparisons, no any move needed.
 - ♦ $O(n)$
- The worst case: reversed order
 - ♦ $n-1$ passes
 - ♦ $(n-1)+(n-2)+\dots+1=n(n-1)/2$ comparisons.
 - ♦ $n(n-1)/2$ movements
 - ♦ $O(n^2)$
- Stable

- `void bubbleSort(int arr[], int n)`
- `{`
- `int i, j;`
- `for (i = 0; i < n - 1; i++)`
- `for (j = 0; j < n - i - 1; j++)`
- `if (arr[j] > arr[j + 1])`
- `swap(arr[j], arr[j + 1]);`
- `}`

Bubble Sort Example and Analysis

A simple example of Bubble Sort

 unsorted

 sorted

 swap

6	11	3	9	4
---	----	---	---	---

An unsorted List with 5 elements

index 1 2 3 4

First pass

Swap 11 and 3

Swap 11 and 9

Swap 11 and 4

At end of the first pass, the largest element is in it's correct position.

6	3	11	9	4
6	3	9	11	4
6	3	9	4	11
6	3	9	4	11

index 0 1 2 3 4

Second Pass

Swap 6 and 3

Swap 9 and 4

At the end of the second pass, the second largest element is in it's correct position

6	3	9	4	11
3	6	9	4	11
3	6	9	4	11
3	6	4	9	11

Bubble Sort Example and Analysis

A simple example of Bubble Sort

<i>index</i>	0	1	2	3	4
	3	6	4	9	11
	3	4	6	9	11
	3	4	6	9	11
<i>index</i>	0	1	2	3	4
	3	4	6	9	11

Third Pass

Swap 6 and 4

At the end of the third pass, the third largest element is in its correct position.

As it happens, the array is now sorted

Fourth Pass

No swaps made as the array is now sorted

 sorted

 swap

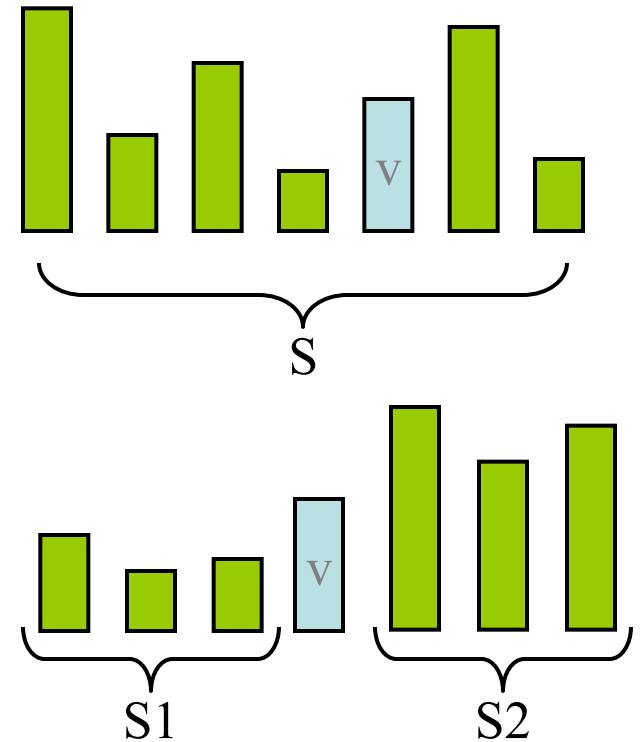
Quicksort

Introduction

- **Fastest** known sorting algorithm in practice
- Average running time: $O(N \log N)$
- Worst case running time: $O(N^2)$
 - ♦ But, the worst case seldom happens.
- Another divide-and-conquer recursive algorithm, like mergesort

Quicksort

- Divide step:
 - ♦ Pick any element (*pivot*) v in S
 - ♦ Partition $S - \{v\}$ into two groups
$$S1 = \{x \in S - \{v\} \mid x \leq v\}$$
$$S2 = \{x \in S - \{v\} \mid x \geq v\}$$
- Conquer step: recursively sort $S1$ and $S2$
- Combine step: the sorted $S1$ (by the time returned from recursion), followed by v , followed by the sorted $S2$ (i.e., nothing extra needs to be done)



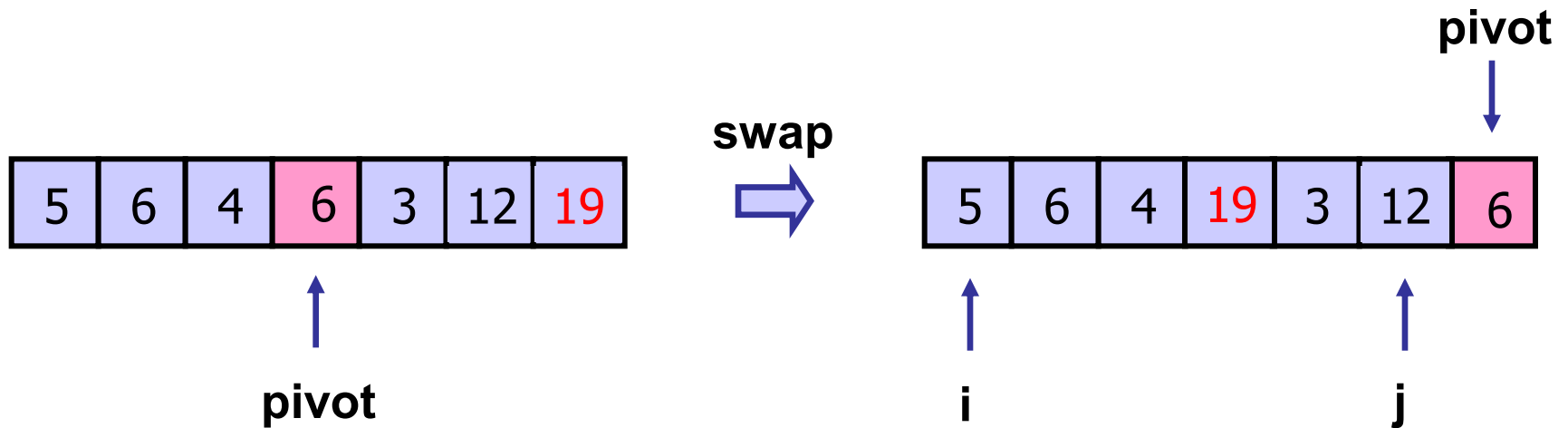
Partitioning

- How to partition
 - ♦ Key step of the quicksort algorithm
 - ♦ Goal: given the picked pivot, partition the remaining elements into two smaller sets
 - ♦ Many ways to implement
 - ♦ Even the slightest deviations may cause surprisingly bad results.
- We will learn an easy and efficient partitioning strategy here.
- How to pick a pivot will be discussed later

Partitioning Strategy

To partition an array $A[\text{left} \dots \text{right}]$

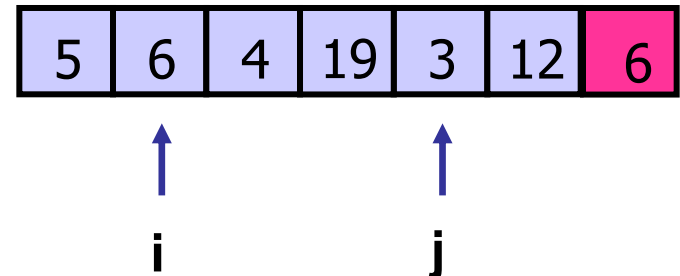
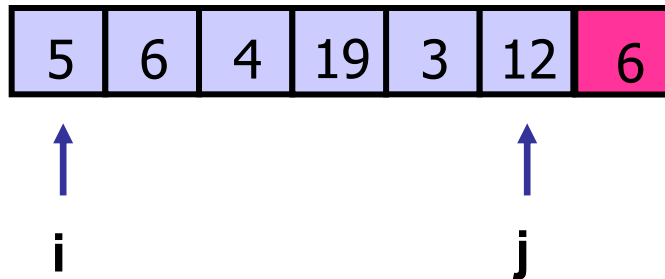
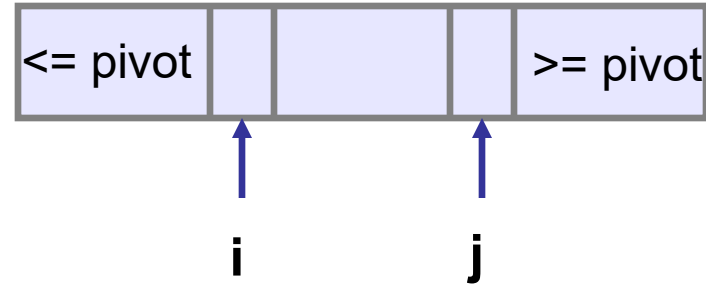
- First, get the pivot element out of the way by swapping it with the last element. (Swap pivot and $A[\text{right}]$)
- Let i start at left and j start at (the last element-1) ($i = \text{left}, j = \text{right}-1$). To partition $A[\text{left} \dots \text{right}-1]$.



Partitioning Strategy

Reach 2 requirements

- ♦ $A[p] \leq \text{pivot}$, for $p < i$
- ♦ $A[p] \geq \text{pivot}$, for $p > j$
- When $i < j$

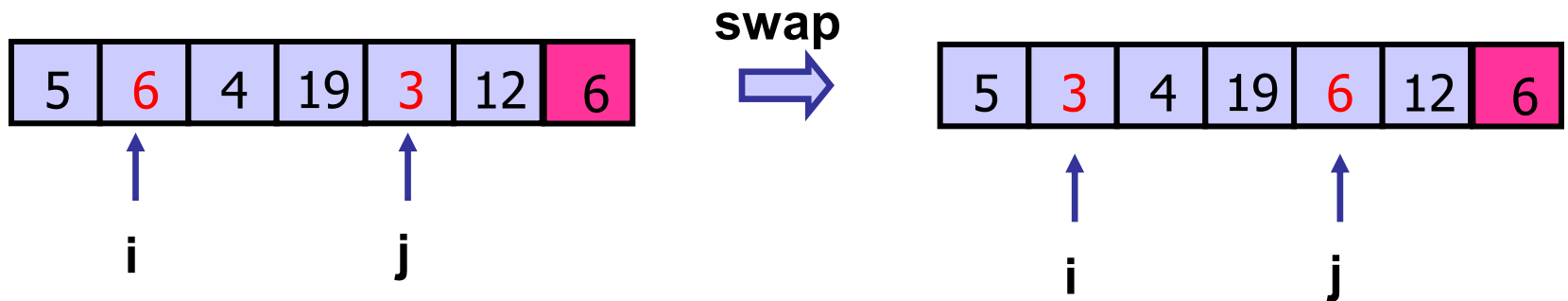


Stops when $A[i] \geq \text{pivot}$

Stops when $A[j] \leq \text{pivot}$

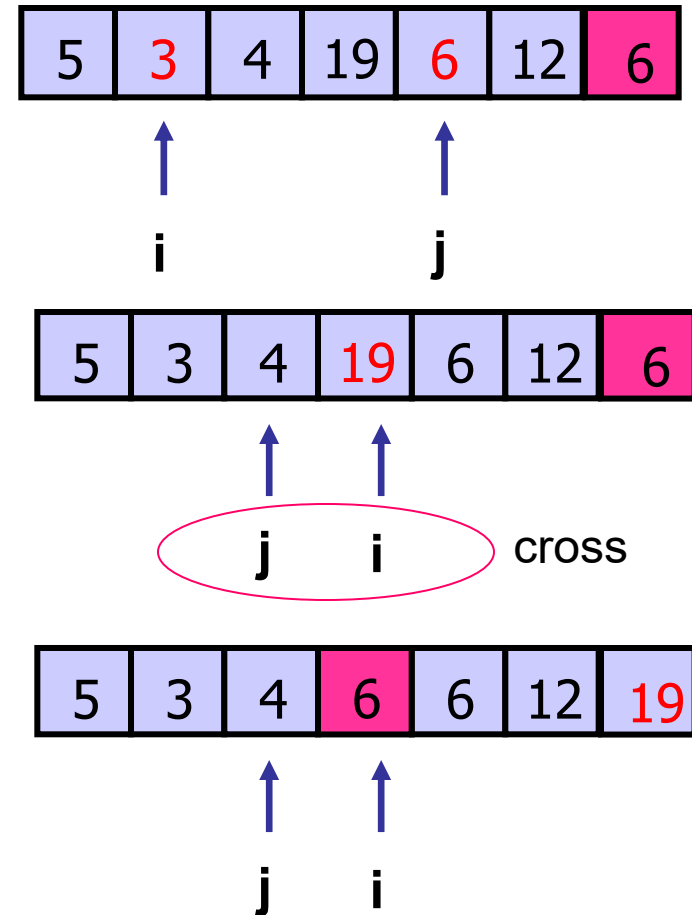
Partitioning Strategy

- When i and j have stopped and i is to the left of j



Partitioning Strategy

- When i and j have crossed
- Results:
 - ♦ $A[p] \leq \text{pivot}$, for $p < i$
 - ♦ $A[p] \geq \text{pivot}$, for $p > i$



Pivot: median of three

```
/* Return median of Left, Center, and Right */  
/* Order these and hide the pivot */
```

ElementType

Median3(ElementType A[], int Left, int Right)

```
{  
    int Center = ( Left + Right ) / 2;  
  
    if( A[ Left ] > A[ Center ] )  
        Swap( &A[ Left ], &A[ Center ] );  
    if( A[ Left ] > A[ Right ] )  
        Swap( &A[ Left ], &A[ Right ] );  
    if( A[ Center ] > A[ Right ] )  
        Swap( &A[ Center ], &A[ Right ] );  
  
    /* Invariant: A[ Left ] <= A[ Center ] <= A[ Right ] */  
  
    Swap( &A[ Center ], &A[ Right - 1 ] ); /* Hide pivot */  
    return A[ Right - 1 ];                /* Return pivot */  
}
```

Pivot: median of three

2	5	6	4	13	3	12	19	6
---	---	---	---	----	---	----	----	---

2	5	6	4	6	3	12	19	13
---	---	---	---	---	---	----	----	----

Ordered

2	5	6	4	6	3	12	19	13
---	---	---	---	---	---	----	----	----



pivot

2	5	6	4	19	3	12	6	13
---	---	---	---	----	---	----	---	----

Swap pivot and A[right - 1]



pivot

Note we only need to partition A[left + 1, ..., right - 2].

Main Quicksort Routine

```
#define Cutoff ( 3 )

void
Qsort( ElementType A[ ], int Left, int Right )
{
    int i, j;
    ElementType Pivot;

    if( Left + Cutoff <= Right )
    {
        Pivot = Median3( A, Left, Right );
        i = Left; j = Right - 1;
        for( ; ; )
        {
            while( A[ ++i ] < Pivot ){ }
            while( A[ --j ] > Pivot ){ }
            if( i < j )
                Swap( &A[ i ], &A[ j ] );
            else
                break;
        }
        Swap( &A[ i ], &A[ Right - 1 ] ); /* Restore pivot */

        Qsort( A, Left, i - 1 );
        Qsort( A, i + 1, Right );
    }
    else /* Do an insertion sort on the subarray */
        InsertionSort( A + Left, Right - Left + 1 );
}
```

Analysis

- Assumptions:
 - ♦ A random pivot (no median-of-three partitioning)
 - ♦ No cutoff for small arrays
- Running time
 - ♦ pivot selection: constant time, i.e. $O(1)$
 - ♦ partitioning: linear time, i.e. $O(N)$
 - ♦ running time of the two recursive calls
- $T(N)=T(i)+T(N-i-1)+cN$ where c is a constant
($T(N)$ denote the running time of quicksort to sort N numbers, $T(0)=T(1)=1$, as in mergesort.)
 - ♦ i : number of elements in S_1

Worst-Case Analysis

- What will be the worst case running time?
 - ♦ The pivot is the smallest element, all the time
 - ♦ Partition is always **unbalanced (i=0)**

$T(N) = T(i) + T(N-i-1) + cN$ where c is a constant

($T(N)$ denote the worst-case running time of quicksort to sort N numbers, $T(0) = T(1) = 1$, as in mergesort.)

i : number of elements in S_1

$$T(N) = T(N-1) + cN \quad (\text{Ignore } T(0)=1)$$

$$T(N-1) = T(N-2) + c(N-1)$$

$$T(N-2) = T(N-3) + c(N-2)$$

$\vdots$

$$T(2) = T(1) + c(2)$$

$$T(N) = T(1) + c \sum_{i=2}^N i = O(N^2)$$

Best-Case Analysis

- What will be the best case running time?
 - ♦ Partition is perfectly balanced.
 - ♦ Pivot is always in the middle (median of the array)

$$T(N) = 2T(N/2) + cN$$

$$\frac{T(N)}{N} = \frac{T(N/2)}{N/2} + c$$

$$\frac{T(N/2)}{N/2} = \frac{T(N/4)}{N/4} + c$$

$$\frac{T(N/4)}{N/4} = \frac{T(N/8)}{N/8} + c$$

⋮

$$\frac{T(2)}{2} = \frac{T(1)}{1} + c$$

$$\frac{T(N)}{N} = \frac{T(1)}{1} + c \log N$$

$$T(N) = cN \log N + N = O(N \log N)$$

$T(N) = T(i) + T(N-i-1) + cN$ where c is a constant

($T(N)$ denote the worst-case running time of quicksort to sort N numbers,

$T(0) = T(1) = 1$, as in mergesort.)

i : number of elements in S_1

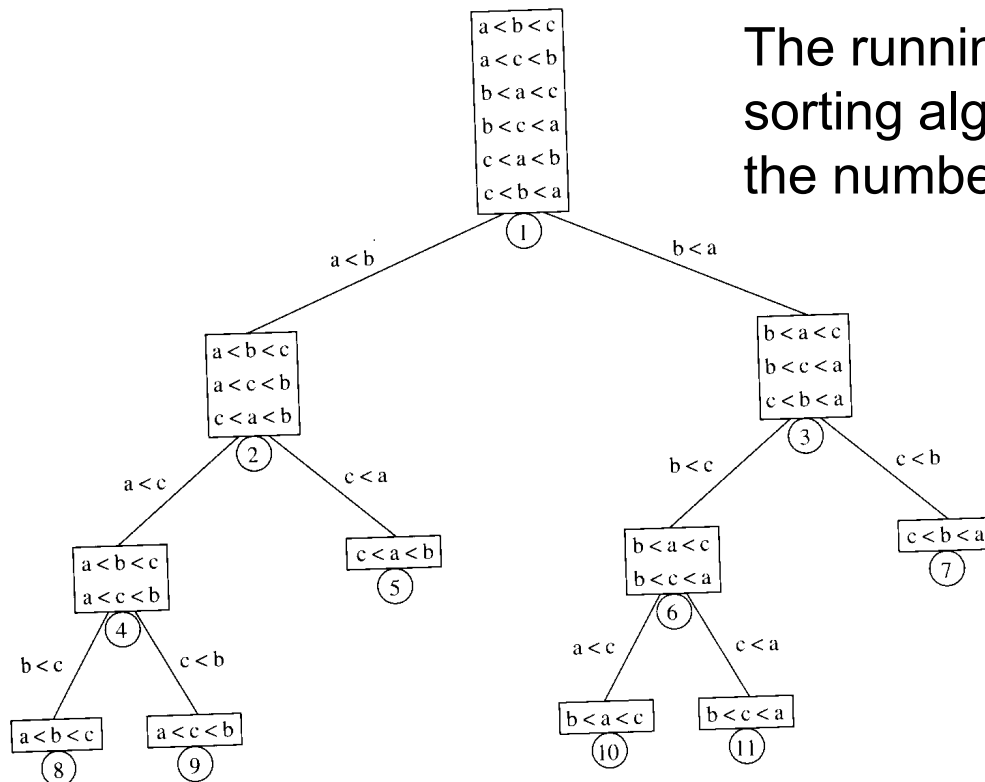
- Average-Case: $O(N \log N)$

Lower Bound for Comparison-based Sorting

Lower Bound for Comparison-based Sorting

- A decision tree is an abstraction used to prove lower bounds.
- Any comparison-based sorting algorithm can be represented by a binary decision tree.

<https://www.youtube.com/watch?v=0ufNJSWOTqo&t=614s>



The running time of any comparison-based sorting algorithm is $\Omega(N \log(N))$, where N is the number of elements to be sorted.

Non-comparison Based Sorting

Bucket Sort

- Assume N integers are to be sorted, each is in the range between 1 and M .
- Define an array $B[1..M]$, initialize all to 0 $\Rightarrow O(M)$
- Scan through the input list A_i , insert A_i into $B[A_i]$ $\Rightarrow O(N)$
- Scan B once, read out the nonzero integers $\Rightarrow O(M)$

Running time: $O(M + N)$

- ♦ If the range size, M , is $O(N)$, then the running time is $O(N)$.
- ♦ It can be bad if the range is very long, e.g. $O(N^2)$. The running is then $O(N^2)$.

$N=7$, $M = 9$,

We want to sort: 8 1 9 5 2 6 3

1	2	3		5	6		8	9		
---	---	---	--	---	---	--	---	---	--	--

Output: 1 2 3 5 6 8 9

Bucket Sort

- What if we have duplicates?
- B is an array of pointers.
- Each position in the array has 2 pointers: head and tail, for a linked-list. Tail points to the end of a linked list, and head points to the beginning.
- $A[j]$ is inserted at the end of the list $B[A[j]]$
- Again, Array B is sequentially traversed and each nonempty list is printed out.
- Time: $O(M + N)$

$M = 9,$

Wish to sort: 8 5 1 5 9 5 6 2 7

1	2			5	6	7	8	9		
---	---	--	--	---	---	---	---	---	--	--

5

5

Output: 1 2 5 5 5 6 7 8 9

Radix Sort

- Bucket sort, the input is a bunch of elements in a small range.
- Radix sort

Extra information: every integer can be represented by at most k digits

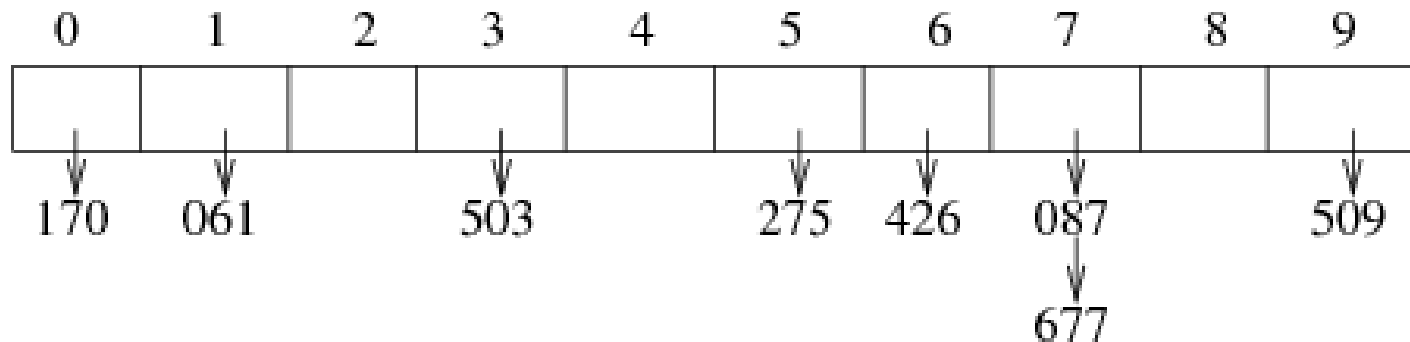
- ♦ $d_1d_2\dots d_k$ where d_i are digits in base r
- ♦ d_1 : most significant digit
- ♦ d_k : least significant digit

Radix Sort Example

- Least-significant-digit-first

Example: 275, 087, 426, 061, 509, 170, 677, 503

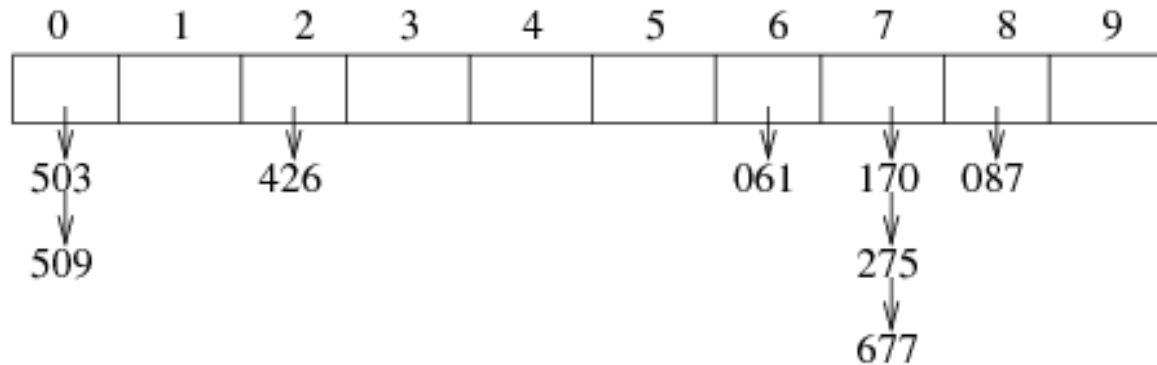
1st pass



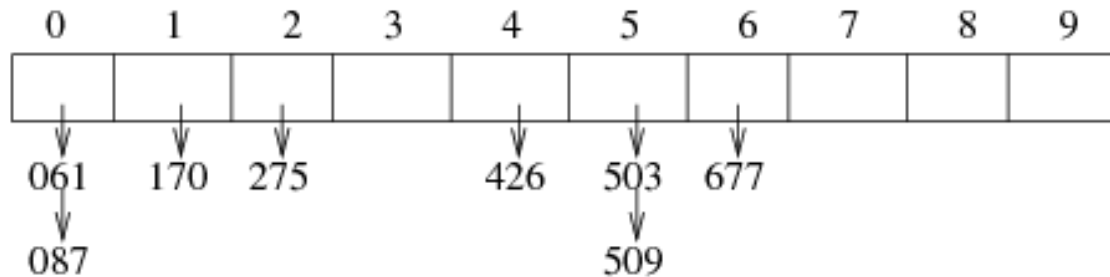
170 061 503 275 426 087 677 509

Radix Sort Example

2nd pass 170 061 503 275 426 087 677 509



3rd pass 503 509 426 061 170 275 677 087



061 087 170 275 426 503 509 677

→
in sorted order

Radix Sort

- Running time
 - ♦ k = number of digits, r = base, N = number of input integers.
 - ♦ k passes: through the whole numbers (i.e. k bucket sorts, with range being $0..(r-1)$).
 - ♦ Each pass takes $O(N)+O(N) = O(N)$ (bucket sort).
 - ♦ Construct a new ordered: $O(r)$. Using the linked list.
 - ♦ Total: $O(k(N+r))$.
 - ♦ If k and r are constants, running time of the Radix Sort is $O(N)$.
- Notes:
 - ♦ Radix sort is not based on comparisons; the values are used as array indices.

Sorting Algorithms Summary

- Comparison based sorting
 - ♦ Insertion sort
 - Insertion sort (direct)
 - Shell sort
 - ♦ Selection sort
 - Heap sort
 - ♦ Merge sort
 - ♦ Exchange sort
 - Bubble sort
 - Quick sort
- Worst case lower bound for comparison-based sorting
- Non-comparison based sorting
 - ♦ Bucket sort
 - ♦ Radix sort